Documentación de Proyecto DataSkins

Introducción

DataSkins es un proyecto de mercado de armas y camuflajes virtuales diseñado con el lenguaje de programación Python 3. En esta documentación se detallará la instalación y uso del programa, ademas del funcionamiento del código.

Descripción

El proyecto consiste en un mercado virtual de armas y camuflajes. Al ingresar, se le ofrece al usuario la opción de iniciar sesión o registrarse. En caso de optar por el registro, el usuario debe proporcionar un nombre de usuario, correo electrónico y contraseña, los cuales son almacenados para futuros accesos. Si elige iniciar sesión, se le solicitan el nombre de usuario y la contraseña, y se comprueba la validez de dichos datos. En caso de olvido de la contraseña, se brinda la opción de cambiarla mediante la verificación del correo electrónico, simulando un proceso de autenticación en dos pasos.

Instalación

Para el uso del programa se debe tener pre-instalado Python en Visual Studio y tener de almacenamiento 1 MB (También se puede ejecutar en la terminal).

A continuación se dará los pasos de la instalación del programa.

1) Descargar el archivo DataSkins.zip
2) Luego Descomprimir el archivo zip
3) Abrir el editor de código de su preferencia (Ej. Visual Studio code), y seleccionar el archivo menu_principal.py de la carpeta DataSkins.
* Como alternativa, abrir la terminal y escribir python y la ruta de menu_principal.py

Contenido

Una vez que el usuario haya iniciado sesión correctamente, dispone de las siguientes opciones:

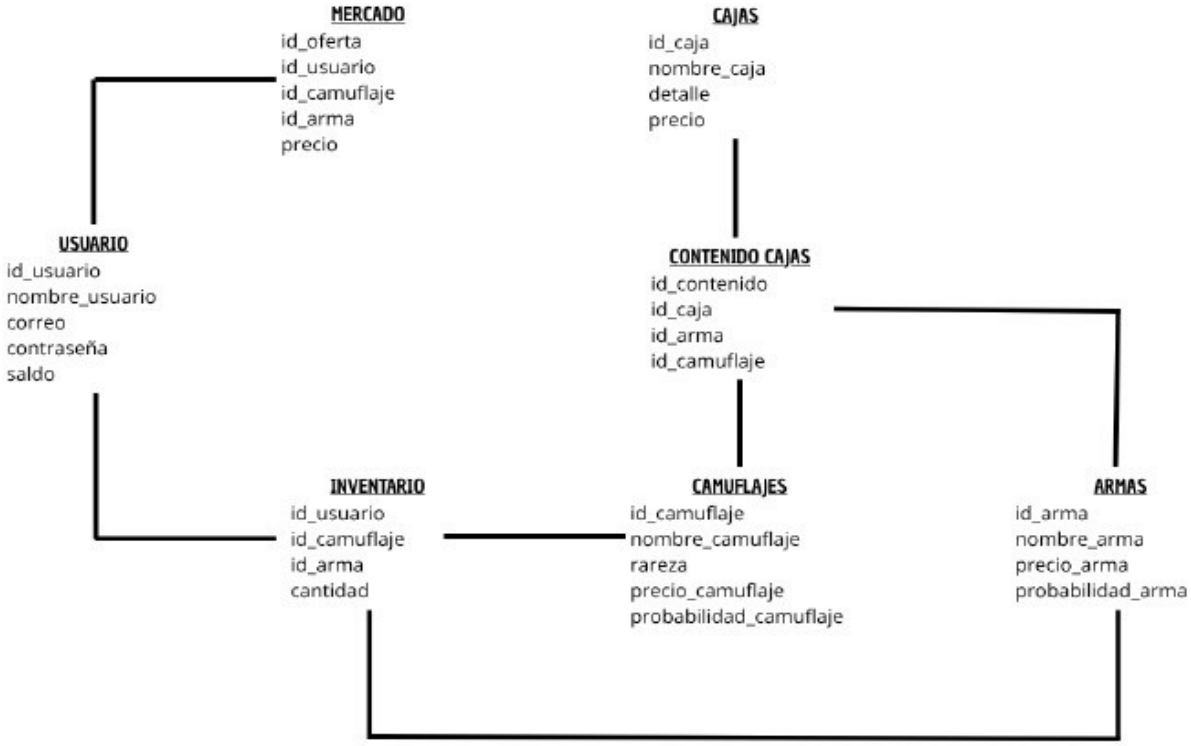
1. **Ingreso de Dinero:** El usuario puede añadir saldo mediante un sistema ficticio de métodos de pago con tarjeta de crédito/débito.

2. **Tienda de Cajas:** Se presenta un listado de cajas, cada una con sus respectivos precios. Cada caja contendrá una selección de armas y, para cada arma, una lista de camuflajes asociados. Los camuflajes estarán clasificados en categorías: común, poco común, raro, épico, mítico y legendario. Al abrir una caja, el sistema utiliza la generación de números aleatorios para determinar el objeto que se recompensa al usuario. Cada arma y camuflaje tiene un precio individual, el cual se le suma para obtener el valor total del premio.

3. **Inventario del Usuario:** El usuario puede consultar un inventario que contiene todos los datos relacionados con sus armas y camuflajes adquiridos. Este inventario permite ordenar los artículos por precio (de menor a mayor o viceversa) y por rareza. Asimismo, se incluye un sistema de búsqueda que muestra los resultados en función de la similitud con el término ingresado por el usuario.

4. **Mercado de Camuflajes:** En esta sección, el usuario tiene la posibilidad de vender y comprar armas y camuflajes de otros usuarios, facilitando así el intercambio entre participantes del mercado.

Diagrama de entidad-relación (DER)



Módulos

A continuación estaremos nombrando la descripción y funcionalidad de cada módulo del código.

menu_principal.py
Es el punto de entrada principal del sistema. Contiene el menú desde el cual el usuario accede a las demás funciones. Importa los módulos de gestión de usuarios, mercado, inventarios y utilidades.

gestion_inventarios.py
Controla los ítems obtenidos por el usuario (armas, camuflajes, cajas, etc.). Permite ver y filtrar inventarios.

gestion_mercado.py
Simula un mercado donde los usuarios pueden comprar y vender ítems. Interactúa con los archivos de inventario y usuarios.

apertura_cajas.py
Contiene la lógica para abrir cajas y obtener recompensas aleatorias. Utiliza diccionarios de probabilidad y datos cargados desde los archivos '.dat'.

enciclopedia.py
Muestra información general de los objetos disponibles en el juego (armas y camuflajes). Es una referencia informativa.

modo_admin.py
Permite al administrador realizar tareas especiales: agregar ítems, modificar precios o controlar el mercado.

utilidades.py
Incluye funciones comunes para manejo de archivos, lectura de datos, validaciones y operaciones repetitivas utilizadas en todos los demás módulos.

Archivos .dat (carpeta 'diccionarios')

Estos archivos guardan información persistente del sistema, en forma de diccionarios almacenados.

• **usuarios.dat**: datos de usuarios registrados.

• **sesion.dat**: información de sesión activa.

• **inventarios.dat**: ítems obtenidos por cada usuario.

• **mercado.dat**: datos del mercado y precios.

• **armas.dat, cajas.dat, camuflajes.dat**: definen el contenido disponible en el sistema.

Relaciones entre archivos

• **menu_principal.py** es el centro del sistema: llama a los módulos de usuarios, mercado e inventarios.

• **gestion_usuarios.py** conecta con los datos de sesión y permite al menú principal reconocer quién usa el sistema.

• **gestion_inventarios.py** y **apertura_cajas.py** trabajan juntos: al abrir una caja, los objetos se agregan al inventario.

• **gestion_mercado.py** se comunica con inventarios y usuarios para actualizar compras o ventas.

• **utilidades.py** es usado por casi todos los módulos.

• **modo_admin.py** puede modificar todos los demás recursos del sistema.

Funciones de utilidades.py

En esta sección detallaremos el comportamiento de las funciones.

• **limpiar():** Utiliza la librería del sistema operativo y ejecuta un comando para limpiar la consola.

• **_leer_tecla_windows():** Lee la entrada por teclado del usuario en windows.

• **gestion_menu():** Establece en un bucle una interfaz que toma la entrada del usuario reutilizable para los demás menús

• **mostrar_menu():** Muestra la interfaz con opciones en pantalla al usuario.

• **mostrar_menu_cajas():** Es similar a **mostrar_menu()** con la diferencia de que toma el diccionario de cajas, las ordena y las muestra.

• **mostrar_menu_filtrado():** Es similar a **mostrar_menu()** con la diferencia de que toma el inventario del usuario, lo filtra y lo muestra en tiempo real.

• **mostrar_menu_enciclopedia():** Es similar a **mostrar_menu()** con la diferencia de que toma los diccionarios de armas y camuflajes y aplica los filtros en tiempo real.

• **cargar_diccionario():** Carga el diccionario según el nombre ingresado accediéndolo desde la carpeta de diccionarios.

• **guardar_diccionario():** Guarda el diccionario según el nombre ingresado accediéndolo desde la carpeta de diccionarios.

• **reservacion_de_espacio():** Toma un diccionario a ordenar y le concatena espacios según el tamaño mayor para que queden todos los elementos de la interfaz alineados.